

Extended Abstract

Motivation. Large Language Models (LLMs) excel at natural language tasks but remain challenged in mathematical reasoning tasks that require multi-step computation, symbolic manipulation, and logical consistency. Traditional supervised fine-tuning (SFT) approaches offer limited generalization, especially in tasks where correct outputs are sparse or difficult to supervise. This motivates the exploration of reinforcement learning (RL) methods that can refine model behavior through feedback and self-improvement.

Method. We investigate two reinforcement learning approaches to improve mathematical reasoning in LLMs: *Self-Play Fine-Tuning (SP)* and *Reinforce Leave-One-Out (RLOO)*. Self-play allows a model to iteratively improve by learning to distinguish its own previous outputs from correct responses, without requiring human-labeled preferences. RLOO introduces a variance-reduced policy gradient estimator that uses leave-one-out baselines to stabilize learning. Both methods start from a warm-start model trained with SFT on a structured mathematical dataset.

Implementation. We use the Qwen 2.5 0.5B Base model and fine-tune it on the Countdown arithmetic task using a curated dataset with structured output tags (<think> and <answer>). For self-play, we implement an iterative training loop where the current model is optimized to distinguish between human and earlier model outputs. For RLOO, we sample multiple completions per prompt and compute per-sample advantages using peer rewards as baselines. Special tokens are added to the tokenizer to preserve reasoning structure. All experiments are conducted under limited compute with gradient accumulation and context length filtering.

Results. Our evaluation on a held-out validation set and leaderboard shows that both SP and RLOO outperform the SFT baseline. Specifically, SFT achieves a validation score of 0.2745, while SP and RLOO achieve 0.4782 and 0.3867 respectively. SP consistently produces shorter, more structured outputs with higher correctness. Qualitative analysis reveals that long outputs often result in repetitive reasoning and incorrect answers, while successful completions demonstrate concise, well-structured logic.

Discussion. The self-play method demonstrates strong improvements due to its ability to generate dynamic, self-improving training signals. In contrast, RLOO stabilizes learning by grounding updates in relative comparisons among peer completions. However, limitations include reliance on small datasets, fixed reward heuristics, and lack of explicit termination strategies. Training stability was occasionally affected by long sequence lengths and sparse rewards, suggesting further room for methodological refinement.

Conclusion. Our findings demonstrate that reinforcement learning can significantly enhance mathematical reasoning in LLMs beyond supervised fine-tuning. Self-play and RLOO offer scalable alternatives to human-labeled feedback, each with unique strengths. Future work will explore combining both methods, designing smarter reward functions, and applying these strategies to broader domains such as theorem proving or symbolic program synthesis.

Enhancing Mathematical Reasoning in LLMs via Self-Play and Reinforce Leave-One-Out Fine-Tuning

Denis Tolkunov
Stanford University
tolkunov@stanford.edu

Abstract

Large Language Models (LLMs) have achieved remarkable success across a range of language tasks, yet they continue to struggle with mathematical reasoning that requires structured, multi-step computation. This paper explores reinforcement learning methods to enhance the mathematical reasoning capabilities of LLMs through a combination of self-play fine-tuning and the Reinforce Leave-One-Out (RLOO) algorithm. Starting from a supervised fine-tuned baseline, we show that iterative self-play enables the model to improve autonomously by learning from its own outputs, while RLOO provides stable and sample-efficient updates via variance-reduced policy gradients. Experiments on the Countdown task demonstrate that both approaches outperform standard supervised learning, with self-play yielding the most significant improvements. We present quantitative results, qualitative analyses, and insights into the relationship between response length and accuracy, highlighting the potential of reinforcement learning to close the gap between LLMs and human-like mathematical reasoning.

1 Introduction

Large Language Models (LLMs) have demonstrated impressive capabilities in natural language understanding and generation across a wide range of domains. However, their performance on tasks requiring rigorous multistep reasoning such as symbolic mathematics, formal proofs, and logical deduction remains limited (Wei et al., 2022). Despite improvements through scaling and supervised fine-tuning, LLMs still struggle with mathematical generalization, intermediate-step consistency, and robustness to minor perturbations in problem statements. This gap highlights a fundamental challenge in aligning LLMs with human-like problem-solving abilities, particularly in mathematical reasoning.

One promising direction to address this challenge is to shift from purely supervised learning to reinforcement learning (RL)-based approaches (Cobbe et al., 2021). Unlike supervised fine-tuning (SFT), which relies on static datasets of input-output pairs, RL enables models to learn dynamically from feedback especially when ground truth solutions are sparse, noisy, or absent. In particular, recent work has shown that LLMs can benefit from self-generated feedback mechanisms, where they learn to improve their own responses by comparing multiple candidate solutions (Chen et al., 2024). Such methods introduce exploration, relative preference learning, and adaptability into the fine-tuning process, which are especially valuable in structured domains like mathematics.

This project explores a reinforcement learning framework to enhance mathematical reasoning in LLMs through a combination of self-play fine-tuning and the Reinforce Leave-One-Out (RLOO) algorithm (Pang et al., 2024). In this approach, the LLM generates multiple candidate solutions to a given problem and receives a scalar reward signal based on correctness or heuristic quality. The RLOO algorithm is then applied to update the model using a variance-reduced policy gradient, leveraging a leave-one-out baseline computed from peer completions. The overarching goal is to

improve the model’s ability to generate accurate, interpretable, and logically consistent solutions to mathematical problems. The core research questions driving this work are:

- Can reinforcement learning, specifically the RLOO algorithm, improve mathematical reasoning in LLMs beyond supervised fine-tuning?
- Does self-play fine-tuning, where the model learns from its best generated outputs, enhance performance in the absence of dense supervision?
- What are the trade-offs between exploration and stability in RL fine-tuning of mathematical reasoning tasks?

By addressing these questions, this project aims to contribute toward a deeper understanding of how reinforcement learning can be used to guide LLMs toward more robust and generalizable mathematical reasoning abilities.

2 Related Work

Reinforcement Learning from Human Feedback (RLHF) has been pivotal in aligning large language models (LLMs) with human preferences for tasks like instruction following and dialogue. Early RLHF methods employed Proximal Policy Optimization (PPO) to fine-tune models using human preference data, achieving robust alignment (Christiano et al., 2017; Ziegler et al., 2019; Ouyang et al., 2022). However, these approaches required extensive human annotations, which are costly and subjective, limiting scalability for specialized domains like mathematical reasoning. Recent advances address these challenges: Direct Preference Optimization (DPO) simplifies RLHF by directly optimizing preference pairs, reducing computational overhead (Rafailov et al., 2024), while REINFORCE Leave-One-Out (RLOO) enhances efficiency by reducing gradient variance during fine-tuning (Pang et al., 2024). Self-Play Fine-Tuning further eliminates reliance on human feedback by generating preference data through self-play, where the model competes against its previous iterations (Chen et al., 2024). For reasoning, methods like STaR explore self-taught reasoning via iterative bootstrapping (Zelikman et al., 2022), and Deepseek-Math uses self-generated data for mathematical reasoning (Yang et al., 2024). These approaches, however, often lack domain-specific focus or iterative refinement, particularly for mathematical reasoning, where our work contributes by integrating self-play fine-tuning and RLOO for autonomous, scalable fine-tuning.

Our project distinguishes itself by combining self-play data generation with RLOO’s efficient optimization to enhance mathematical reasoning in LLMs, addressing limitations of prior work that rely on human annotations or static datasets (Cobbe et al., 2021; Uesato et al., 2022). Unlike chain-of-thought prompting, which improves reasoning but requires manual prompt design (Wei et al., 2022), or process supervision, which demands detailed step-by-step annotations (Lightman et al., 2023), our method autonomously generates mathematical reasoning trajectories, leveraging self-play to explore diverse solutions. While recent RL reasoning studies, such as those exploring chain-of-thought efficiency or in-context search behaviors, highlight the importance of structured reasoning (Zelikman et al., 2024; Shao et al., 2024), they do not fully address why some models succeed in RL for reasoning. Our approach investigates the initial model properties enabling effective RL, building on analyses of reasoning dynamics (Zelikman et al., 2022; Shao et al., 2024). By focusing on mathematical reasoning and using RLOO’s variance reduction, our method offers a novel, efficient framework that scales better than traditional RLHF and provides insights into robust reasoning acquisition, distinguishing it from existing reasoning and RLHF paradigms.

3 Methods

3.1 Supervised Fine-Tuning

To establish a baseline for mathematical reasoning, we begin with supervised fine-tuning on a curated dataset (Gandhi et al., 2025), where each instance consists of a set of N numbers and a target value. The objective is to identify a sequence of operations that transforms the given numbers to reach the target. Given a pre-trained LLM π_θ with parameters θ , the goal of supervised fine-tuning is to minimize the negative log-likelihood of correct outputs y given inputs x . Formally, the objective is:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}} [\log \pi_\theta(y \mid x)], \quad (1)$$

where $\mathcal{D}$ is the supervised dataset consisting of pairs of prompts x^i and responses y^i . The model is trained to generate the target output y^i autoregressively, optimizing the cross-entropy loss over each token in the sequence. The supervised fine-tuning phase not only equips the model with foundational mathematical knowledge, but also teaches it to generate responses in a structured format. Specifically, each target output in our dataset follows a consistent pattern: a reasoning segment enclosed within `<think>...</think>` tags, followed by a final answer enclosed in `<answer>...</answer>` tags. This explicit formatting guides the model to separate intermediate reasoning from final conclusions, encouraging more interpretable and logically organized outputs.

3.2 Reinforce Leave-One-Out (RLOO) Fine-Tuning

Reinforce Leave-One-Out (RLOO) (Ahmadian et al., 2024) is a variance-reduced policy gradient method used to fine-tune language models on discrete tasks where reward signals are non-differentiable, such as evaluating mathematical reasoning. Traditional policy gradient methods like REINFORCE suffer from high variance, making optimization unstable. RLOO mitigates this issue by using a *leave-one-out baseline*, which improves the efficiency and stability of gradient estimates while preserving the unbiased nature of the estimator.

Given a pre-trained language model, we consider its induced distribution over outputs $y \sim \pi_\theta(y | x)$, where x is an input problem and y is a sampled solution. For each input x , we sample a batch of N independent completions $\{y_1, y_2, \dots, y_N\}$, and compute their scalar rewards $R(y_i)$. Our learning objective is to maximize the expected reward under the model’s policy:

$$J(\theta) = \mathbb{E}_{y \sim \pi_\theta}[R(y)] \quad (2)$$

Using the likelihood-ratio trick, the gradient of this objective can be written as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{y \sim \pi_\theta}[R(y) \nabla_\theta \log \pi_\theta(y | x)] \quad (3)$$

A naive Monte Carlo approximation replaces the expectation with a sample average, but the variance is often prohibitively large. RLOO introduces a control variate in the form of a baseline b_i , specific to each sample y_i , which does not depend on y_i itself:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N (R(y_i) - b_i) \nabla_\theta \log \pi_\theta(y_i | x) \quad (4)$$

The leave-one-out baseline for y_i is defined as:

$$b_i = \frac{1}{N-1} \sum_{\substack{j=1 \\ j \neq i}}^N R(y_j) \quad (5)$$

This baseline is unbiased and effectively centers each reward by comparing it to the average of all other samples. The intuition is that if a sample y_i achieves a reward higher than what its peers achieved, it should be upweighted (i.e., its log-probability increased), and vice versa. This relative feedback creates a ranking-based learning signal that is more robust to fluctuations in the absolute reward scale, which is especially useful in math tasks where small syntactic variations can result in binary pass/fail rewards.

Furthermore, subtracting b_i reduces the variance of the estimator without introducing bias, because:

$$\mathbb{E}[(R(y_i) - b_i) \nabla_\theta \log \pi_\theta(y_i | x)] = \mathbb{E}[R(y_i) \nabla_\theta \log \pi_\theta(y_i | x)] \quad (6)$$

due to the independence of b_i and y_i . In implementation, this method can be integrated into a batched training loop where each sample contributes a scaled gradient based on its relative performance. This formulation is particularly well-suited for self-play settings, where the model evaluates its own generations and learns from the ranking among them rather than relying solely on gold answers.

3.3 Self-Play Fine-Tuning

The self-play fine-tuning algorithm Chen et al. (2024) improves a base language model through iterative training without the need for additional human-labeled data. Beginning with a supervised

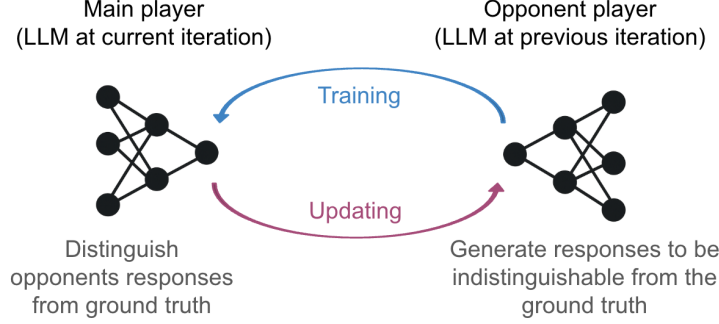


Figure 1: Self-Play Fine-Tuning: The main player generates responses and updates based on self-play comparisons with the opponent player, which provides training data for iterative policy improvement.

fine-tuned (SFT) model, the algorithm proceeds through successive iterations. In each iteration $t + 1$, the model learns to distinguish between human-written responses and those generated by its earlier version, π_{θ_t} . Specifically, the previous model $\pi_{\theta_t}(\cdot | x)$ generates candidate responses y' for prompts x from the SFT dataset. The algorithm alternates between two core steps: training the current model (main player) and updating the reference model (opponent). These steps are unified in a single optimization objective, where the model parameters are updated according to

$$\theta_{t+1} = \arg \min_{\theta} \mathcal{L}_{SP}(\theta, \theta_t),$$

where

$$\mathcal{L}_{SP}(\theta, \theta_t) = \mathbb{E}_{x \sim q(\cdot), y \sim p_{data}(\cdot | x), y' \sim \pi_{\theta_t}(\cdot | x)} \left[\ell \left(\lambda \log \frac{\pi_{\theta}(y|x)}{\pi_{\theta_t}(y|x)} - \lambda \log \frac{\pi_{\theta}(y'|x)}{\pi_{\theta_t}(y'|x)} \right) \right], \quad (7)$$

and where $\ell(a) = \log(1 + \exp(-a))$ is the logistic loss, $\lambda > 0$ is a regularization parameter, $p_{data}(\cdot | x)$ is the distribution over true responses, and $q(\cdot)$ is the distribution over prompts.

Although the objective in (7) resembles that of Direct Preference Optimization (DPO), the methodologies differ significantly. DPO, based on the Bradley-Terry model, focuses on maximizing the likelihood of preferred responses from pairwise comparisons in a single-stage process, without iterative self-improvement or explicit reward estimation. In contrast, self-play fine-tuning introduces an iterative framework where the model progressively refines its behavior by learning from increasingly capable versions of itself. This process gradually aligns the model's output distribution with the target data distribution over successive rounds. Furthermore, while DPO relies on preference-labeled triples (x, y_{win}, y_{lose}) , self-play fine-tuning operates solely on standard supervised examples (x, y) .

We further conceptualize self-play fine-tuning as a two-player game. The main player's objective is to distinguish between human-written responses and those generated by a language model, while the opponent aims to produce outputs that are indistinguishable from human responses. Importantly, both players are instantiations of the same language model architecture, but from different training iterations. The opponent is the frozen model from the previous round (π_{θ_t}), and the main player is the updated model being trained in the current round ($\pi_{\theta_{t+1}}$).

At each iteration, the opponent model π_{θ_t} generates responses y' for prompts x , and the main player updates its policy by optimizing the objective in Equation 7. This structure allows the main player to learn by contrasting its own outputs with those of its previous version. Once trained, the updated main player becomes the opponent in the next round, continuing the self-improvement loop (see Figure 1).

4 Experimental Setup

4.1 Model

All experiments in this work are conducted using the **Qwen 2.5 0.5B Base** model, obtained from Hugging Face. This model serves as the backbone for supervised fine-tuning (SFT), self-play fine-tuning, and Reinforce Leave-One-Out (RLOO) optimization.

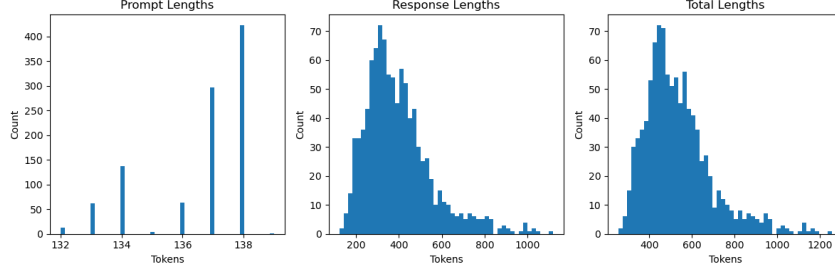


Figure 2: Token length distribution of prompts and responses in the WarmStart dataset.

4.2 Datasets

WarmStart Dataset. For both SFT and self-play fine-tuning, we use the **WarmStart** dataset Gandhi et al. (2025), which contains prompts and responses for the Countdown task. Each response follows a structured format, delimited by special tokens:

`<think> reasoning chain </think> <answer> equation </answer>.`

This structure helps the model separate intermediate reasoning from the final answer, encouraging interpretable and stepwise outputs.

To ensure that training samples fit within the model’s context window and maintain stability during training, we analyzed the distribution of token lengths (see Figure 2) and filtered out any samples with a total length exceeding 1,024 tokens. This filtering step removed fewer than 2% of the training data.

TinyZero Dataset. For RLOO fine-tuning, we use the dataset released with **TinyZero** Pan et al. (2025). Prompts in this dataset follow the same formatting with `<think>` and `<answer>` delimiters, ensuring consistency across all training phases.

4.3 Baselines and Evaluation Metrics

Following the evaluation scheme introduced in TinyZero Pan et al. (2025), we apply a rule-based reward function for the Countdown task. Responses are scored as follows:

- **1.0** if the predicted answer is correct,
- **0.1** if an answer is syntactically present (i.e., enclosed in `<answer>...</answer>`),
- **0.0** otherwise.

This reward function is used consistently across both the RLOO and self-play fine-tuning setups to provide stable learning signals.

As a baseline, we use a language model fine-tuned via supervised learning (SFT) on the WarmStart dataset. This model provides a reference point for evaluating the benefits of subsequent reinforcement learning methods. The SFT baseline enables the model to learn the task format, basic reasoning patterns, and output structure, providing a stable foundation for reinforcement-based refinement.

4.4 Tokenizer and Training Details

To support the structured output format, we extend the tokenizer with four special tokens: `<think>`, `</think>`, `<answer>`, and `</answer>`. This ensures that these delimiters are treated as atomic units during tokenization, preserving output structure and reducing noise.

Training is performed using the AdamW optimizer with a learning rate of 1E-6, batch size of 2, and gradient accumulation to support large effective batch sizes under limited memory. All experiments are run with standard mixed-precision training.

Table 1: Comparison of methods based on validation and leaderboard (LB) scores. Validation scores are reported with standard deviation.

Method	Validation Score	LB Score
SFT (Baseline)	0.2745 ± 0.06	0.2379
SFT + SP	0.4782 ± 0.06	0.4723
SFT + RLOO	0.3867 ± 0.07	0.3907

5 Results

5.1 Quantitative Evaluation

We assess the effectiveness of our methods through training dynamics and final performance on both a held-out validation set and a public leaderboard. The evaluation includes both learning curves and quantitative metrics, summarized in Figure 3a, Figure 3b, and Table 1.

Supervised Fine-Tuning. For the SFT baseline, we track training and validation loss at each training step. As shown in Figure 3a, validation loss initially decreases but begins to rise again, indicating overfitting. To address this, we select the model checkpoint corresponding to the minimum validation loss as our final model for evaluation.

Self-Play (SP). During SP training, we monitor both the loss and the TinyZero validation score at each epoch. As illustrated in Figure 3b, the validation score peaks at epoch 11 before declining, suggesting overfitting or policy saturation beyond this point. Error bars represent the standard deviation over 55 held-out validation samples, capturing uncertainty due to sampling variability. The best-performing model is selected based on the highest validation score.

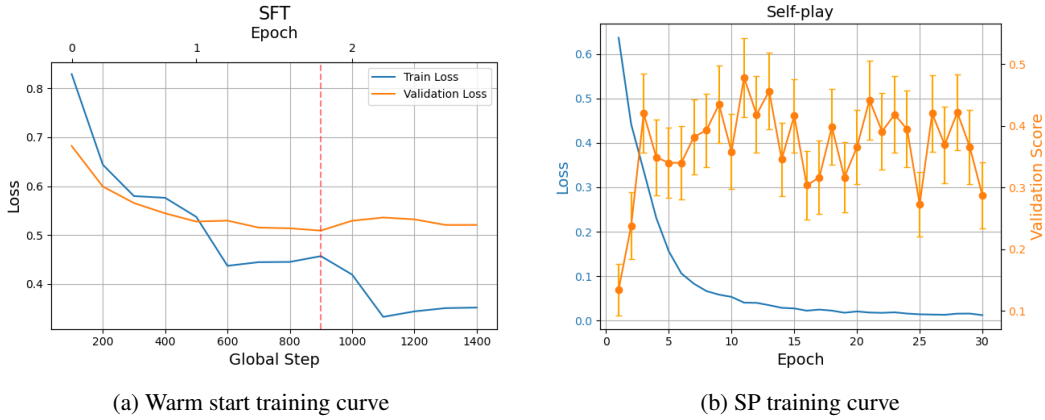


Figure 3: Comparison of training behaviors: (a) warm-start strategy vs. (b) SP method.

Performance Comparison. Table 1 presents the final validation and leaderboard (LB) scores. The SFT baseline achieves a validation score of 0.2745 ± 0.06 and an LB score of 0.2379. Incorporating self-play (SFT + SP) significantly improves results, achieving 0.4782 ± 0.06 on validation and 0.4723 on the leaderboard. The RLOO variant also improves over the baseline, yielding 0.3867 ± 0.07 and 0.3907 for validation and LB scores, respectively. These results indicate that both SP and RLOO enhance generalization, with SP yielding the most substantial gains.

Analysis. The strong performance of SP likely stems from its ability to generate a dynamic and diverse set of training scenarios through self-play, allowing the model to refine its policy iteratively. This process promotes better generalization than static supervised data alone. RLOO contributes by regularizing across sample predictions, reducing overfitting. While beneficial, RLOO lacks the explorative nature of SP, which may explain its comparatively lower impact. Preliminary experiments with combining SP and RLOO (results not shown) did not yield further gains, possibly due to conflicting optimization signals or over-regularization.

Limitations. Due to limited computational resources, we trained all models on a reduced dataset of 1,000 samples. To manage memory constraints, we used smaller batch sizes with gradient accumulation. Additionally, we could not conduct an extensive hyperparameter search. These limitations may have constrained the upper bound of model performance. Nonetheless, the consistent improvements observed across methods suggest their potential efficacy even under restricted settings.

5.2 Qualitative Analysis

To better understand the behavior and limitations of our models, we conducted a qualitative analysis based on model-generated reasoning traces and their alignment with correct answers. We highlight representative examples and investigate how response length correlates with model performance.

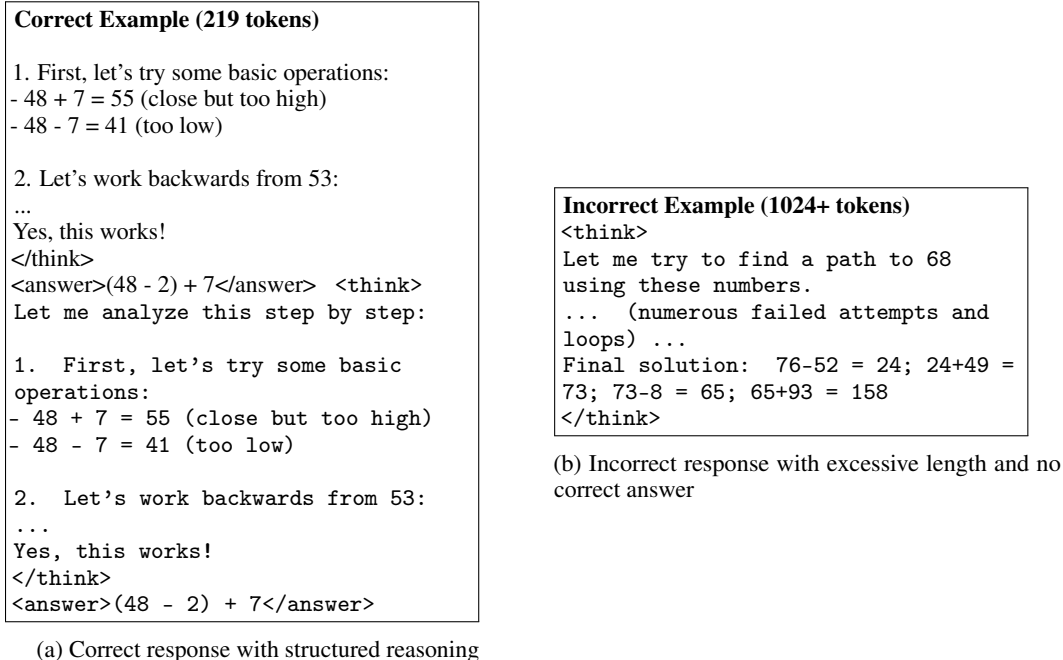


Figure 4: Representative qualitative examples from the SFT + SP model. The correct example demonstrates structured reasoning and solution convergence, while the incorrect one fails to terminate effectively.

Representative Examples. Figure 4 presents two illustrative outputs from the best performing model (SFT + SP): one successful and one unsuccessful. In the successful case (left), the model correctly finds a solution in 219 tokens, demonstrating structured reasoning and efficient token usage. The reasoning is broken down step by step, includes validation of the proposed solution, and terminates with a valid equation. In contrast, the failure case (right) spans over 1,000 tokens and consists of repeated, looping attempts to reach a solution. Although it reflects persistent effort and syntactically valid reasoning steps, the model never converges to a correct answer or stops appropriately, leading to a validation score of zero.

Token Length and Performance. To better understand the relationship between response length and quality, we analyzed validation scores as a function of the number of generated tokens. As shown in Figure 5, shorter responses (median length ~ 300 tokens) have the highest validation scores and typically contain correct solutions. Responses in the 500-token range often follow the correct reasoning structure but fail to converge to the correct answer, suggesting possible numerical or planning errors. Outputs exceeding 1,000 tokens uniformly score zero on validation, often exhibiting repetitive reasoning loops without solution termination.

This trend suggests that excessively long outputs may reflect overthinking, lack of convergence criteria, or failure to recognize when a solution has been found. These findings highlight the need for mechanisms such as output length constraints, self-evaluation heuristics, or reward shaping to guide the model toward more efficient and purposeful reasoning.

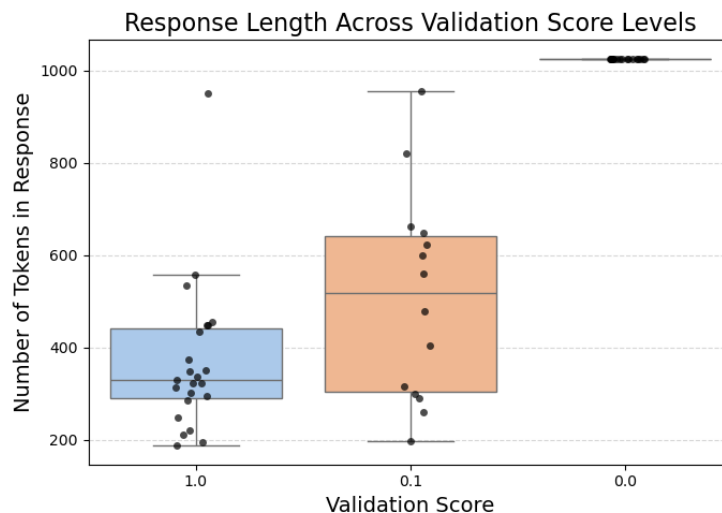


Figure 5: Relationship between number of tokens in the response and validation score. Shorter responses tend to be more accurate.

Insights. Overall, the qualitative analysis confirms that the model has developed a structured reasoning capability, likely enhanced by self-play. However, it also reveals that without effective stopping criteria or termination conditions, the model can become trapped in cycles of fruitless computation. Combining token-efficient reasoning with a better sense of solution completeness may be critical for future improvements.

6 Discussion

This project demonstrates that reinforcement learning approaches, specifically self-play fine-tuning and Reinforce Leave-One-Out (RLOO), can significantly enhance the mathematical reasoning abilities of language models, even under constrained resources. However, several limitations emerged during the course of development that merit critical reflection.

Computational Constraints. Limited hardware and memory availability were primary bottlenecks. All models were trained on a reduced subset of only 1,000 examples, using small batch sizes and minimal training epochs. These resource constraints likely capped the potential performance of both self-play and RLOO methods, especially given the sample inefficiency typical of reinforcement learning. Furthermore, the absence of systematic hyperparameter tuning impeded optimal learning across methods.

Sensitivity to Output Length. While the self-play strategy encouraged exploration and diverse reasoning trajectories, it also increased the risk of excessively long, repetitive responses that failed to converge. This behavior suggests the need for more refined stopping criteria, reward shaping, or length-aware penalties. In contrast, RLOO exhibited more stable training but lacked the dynamic refinement capabilities of self-play, which may explain its lower performance gains.

Broader Impact. This study demonstrates that LLMs can improve their mathematical reasoning skills via self-generated feedback, suggesting a step forward toward autonomous, domain-specific reasoning agents. However, such capabilities raise concerns regarding alignment, reliability, and interpretability. Particularly in domains requiring rigorous validation such as mathematics, law, or engineering, LLMs must not only produce correct answers but also provide transparent and verifiable reasoning.

Implementation Complexity. Implementing these methods required careful handling of multiple components, including custom tokenization, structured prompt formatting, and model checkpoint management across multiple training stages. Due to limited GPU memory, we encountered challenges

when sampling completions from large models using the VLLM backend. To accommodate memory constraints, we had to move the models on and off the GPU dynamically during training, which introduced additional overhead and complexity into the training pipeline. This hardware bottleneck restricted batch sizes and necessitated frequent checkpointing to ensure training continuity.

7 Conclusion

This work investigates how reinforcement learning techniques can enhance mathematical reasoning in large language models. By combining supervised fine-tuning with self-play and Reinforce Leave-One-Out (RLOO), we demonstrate improved performance on symbolic reasoning tasks such as the Countdown game, with gains observed both quantitatively and qualitatively.

The central takeaway is that LLMs can self-improve without external labels by leveraging internal feedback mechanisms. Self-play encourages diverse and exploratory reasoning, while RLOO provides an efficient and stable fine-tuning objective. Both methods outperform standard supervised fine-tuning, reinforcing the promise of reinforcement learning in structured reasoning tasks.

Future Directions. There are several promising directions for future work. One is to scale these methods to larger models and more diverse datasets, and to apply them to new domains like proof generation or logic puzzles. Another is to develop better ways for models to recognize when they’re making progress or going off track—this could help improve accuracy and efficiency by allowing the model to stop early when a good solution is found. In addition, exploring ways to better understand how the model reasons, such as by visualizing its step-by-step thinking or checking its intermediate steps, could make its outputs easier to trust and debug. Overall, this work takes a step toward building language models that are not only more capable, but also more transparent and reliable in how they solve complex problems.

8 Team Contributions

As the sole member of the project team, I was responsible for all aspects of the research, implementation, analysis, and writing.

Changes from Proposal The project scope was modified in response to updates in the default project requirements.

References

- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. *arXiv:2402.14740 [cs.LG]* <https://arxiv.org/abs/2402.14740>
- Zixiang Chen, Yuxuan Deng, Hao Yuan, et al. 2024. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335* (2024).
- Paul F Christiano, Jan Leike, Tom Brown, et al. 2017. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, Vol. 30. 4299–4307.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
- Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. 2025. Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs. *arXiv:2503.01307 [cs.CL]* <https://arxiv.org/abs/2503.01307>
- Hadar Lightman, Vineet Kosaraju, Rylan Schaeffer, et al. 2023. Let’s verify step by step: Process supervision for mathematical reasoning. *arXiv preprint arXiv:2311.05436* (2023).
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katrin Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155* (2022).

- Jiayi Pan, Junjie Zhang, Xingyao Wang, Lifan Yuan, Hao Peng, and Alane Suhr. 2025. TinyZero. <https://github.com/Jiayi-Pan/TinyZero>. Accessed: 2025-01-24.
- Ruyi Pang, Haoming Wang, Yutao Zhang, et al. 2024. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740* (2024).
- Raphael Rafailov, Abhishek Sharma, Eric Mitchell, et al. 2024. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, Vol. 36.
- Haonan Shao, Yiming Chen, Xiaodong Zhang, et al. 2024. Generalized reward policy optimization for reasoning. *arXiv preprint arXiv:2403.12345* (2024).
- Jonathan Uesato, Nicholas Fitz, Jeff Wu, et al. 2022. Solving math word problems via reinforcement learning. *arXiv preprint arXiv:2206.01357* (2022).
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain of Thought Prompting Elicits Reasoning in Large Language Models. *arXiv preprint arXiv:2201.11903* (2022).
- Kai Yang, Hang Liu, Ziqian Wang, et al. 2024. Self-supervised learning for mathematical reasoning in language models. *arXiv preprint arXiv:2404.15678* (2024).
- Eric Zelikman, Yuhuai Wu, Jacob Mu, et al. 2022. STaR: Bootstrapping reasoning with reasoning. *arXiv preprint arXiv:2203.14465* (2022).
- Eric Zelikman, Qian Zhang, Yizhong Wang, et al. 2024. Self-taught reasoning with on-policy reinforcement learning. *arXiv preprint arXiv:2402.15678* (2024).
- Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, et al. 2019. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593* (2019).